Search – Introduction to Artificial Intelligence (AI)

1. What Is Artificial Intelligence?

Definition

Artificial Intelligence (AI) is the field of computer science that focuses on **making machines behave intelligently**, meaning they can:

- Make decisions
- Solve problems
- Learn from experience
- Understand their environment

Everyday Examples

Below are **common AI examples explained step by step**, focusing on *how AI is used* and *why it is considered AI*.

- **Google Maps choosing the best route** Google Maps uses AI and search algorithms to find the best path from your location to your destination.
    - The **state** is your current location on the map
    - The **actions** are possible roads you can take
    - The **goal** is your destination
    - The **path cost** can be time, distance, or traffic delay Google Maps continuously updates the route using real-time data (traffic, accidents), which makes it an intelligent system.
- **Face recognition on smartphones** Your phone learns what your face looks like using many images.
    - AI extracts important features (eyes, nose, face shape)
    - It compares your face to stored patterns
    - If the similarity is high enough, the phone unlocks This is AI because the system **learns from data** and makes a decision based on probability, not fixed rules.
- **Voice assistants (Siri, Alexa)** Voice assistants use AI to understand spoken language.
    - Speech is converted into text (speech recognition)
    - The text is analyzed to understand intent (natural language processing)
    - The assistant searches for the best response or action This involves AI because human language is ambiguous and complex.
- **Email spam filtering** Email systems automatically classify emails as spam or not spam.
    - The AI looks at words, sender behavior, links, and patterns
    - It learns from previous emails marked as spam
    - Over time, the system improves its decisions This is a classic example of **machine learning**, a core area of AI.

> **note:** which of these systems they personally use and what decisions the AI is making for them.

Course Roadmap: What You Will Learn in This AI Course

In this course, we will study **core areas of Artificial Intelligence**. Each area answers a different question about how intelligent systems work.

---

1. Search

Main question: How can an AI find a solution step by step?

Search is about **exploring possibilities** to reach a goal.

Example: Google Maps

- The AI searches through many possible roads
- It evaluates different paths
- It selects the best route based on time or distance

In this course, search helps us understand how AI **thinks systematically** instead of guessing.

---

2. Knowledge

Main question: How can an AI store facts and reason about them?

Knowledge in AI means:

- Representing information (facts, rules)
- Using logic to draw conclusions

Example: Medical diagnosis system

- Fact: “Fever and cough may indicate flu”
- AI reasons using rules to suggest a diagnosis

We study how AI can **know things and reason**, not just calculate.

---

3. Uncertainty

Main question: How does AI make decisions when it is not sure?

In the real world, information is often **incomplete or noisy**.

Example: Weather prediction

- AI cannot be 100% sure it will rain
- Instead, it says “70% chance of rain”

Here, AI uses **probability** to handle uncertainty intelligently.

---

4. Optimization

Main question: How can AI choose the best option among many?

Optimization is about **finding the best possible solution** under constraints.

Example: Delivery route planning

- Many delivery orders
- Limited fuel and time
- AI finds the route with minimum cost

This topic teaches AI to be **efficient**, not just correct.

---

5. Learning (Machine Learning)

Main question: How can AI improve from experience?

Learning allows AI systems to:

- Learn patterns from data
- Improve performance over time

Example: Email spam detection

- AI learns from emails marked as spam
- It improves classification accuracy

This is how AI systems **adapt and improve automatically**.

---

6. Neural Networks

Main question: How can AI be inspired by the human brain?

Neural networks are models inspired by how neurons work.

Example: Image recognition

- Input: an image
- Neural network learns visual patterns
- Output: “This is a cat”

Neural networks are the foundation of **deep learning**.

---

7. Language (Natural Language Processing)

Main question: How can AI understand human language?

Language is difficult because it is:

- Ambiguous

- Context-dependent

Example: Chatbots and translators

- AI understands user input
- AI generates meaningful responses

This topic explains how AI interacts with humans using language.

2. AI as a Problem-Solving System

AI often works by **solving problems through search**.

Real-Life Analogy

Imagine you are lost in a city and want to reach your destination:

- You start at one place
- You try different roads
- You choose the best path

AI does the same thing, but with **algorithms**.

Examples of Search Problems

Search problems appear whenever an AI must **find a sequence of actions** that leads from a starting situation to a goal situation. Below are common examples explained clearly.

- **Maze solving** The AI starts at the entrance of a maze and must reach the exit.
    - **State:** the current position in the maze
    - **Actions:** move up, down, left, or right (if no wall blocks the way)
    - **Goal:** reach the exit cell
    - **Path cost:** number of steps taken The AI explores different paths until it finds one that reaches the exit. This helps to understand how AI explores alternatives.
- **Navigation (GPS)** This is a real-world search problem used every day.
    - **State:** current geographic location
    - **Actions:** drive along available roads
    - **Goal:** arrive at the destination
    - **Path cost:** travel time, distance, or fuel The AI compares many possible routes and chooses the best one based on the selected cost.
- **Sliding puzzles (e.g., 8-puzzle or 15-puzzle)** The puzzle consists of numbered tiles and one empty space.
    - **State:** current arrangement of tiles
    - **Actions:** slide a tile into the empty space
    - **Goal:** tiles arranged in the correct order
    - **Path cost:** number of moves This example clearly shows how AI searches through many configurations to reach a solution.

> In all these examples, AI does not guess. It systematically searches for a solution

3. Key Concepts and Terminology

Agent

An **agent** is the most fundamental concept in Artificial Intelligence.

An agent is an entity that:

- **Observes** its environment using sensors
- **Decides** what to do based on those observations
- **Acts** upon the environment using actions

In simple words:

> An agent is **something that perceives and acts**.

Why do we need the concept of an agent?

AI systems do not exist in isolation. They always:

- Exist inside an **environment** (real or virtual)
- Interact with that environment over time

The agent concept helps us:

- Clearly define *who* is making decisions
- Separate the **decision-maker (agent)** from the **world (environment)**
- Design intelligent behavior step by step

Agent–Environment Interaction

The interaction works as a continuous loop:

1. The environment provides information (perception)
2. The agent analyzes this information
3. The agent chooses an action
4. The environment changes as a result

This loop repeats continuously.

Examples of Agents (Explained)

- **Robot**
    - **Environment:** room, obstacles, people
    - **Perception:** camera, sensors
    - **Actions:** move, stop, pick objects The robot decides how to move safely and efficiently.

- **Game-playing agent** (e.g., chess, tic-tac-toe)
    - **Environment:** game board
    - **Perception:** current board state
    - **Actions:** legal moves The agent selects moves to win or avoid losing.
- **Car navigation system**
    - **Environment:** road network, traffic conditions
    - **Perception:** GPS position, traffic data
    - **Actions:** suggest turns and routes The agent helps the driver reach the destination efficiently.

Types of Agents

- **Simple reactive agents**: respond directly to the environment (e.g., thermostat)
- **Goal-based agents**: act to achieve a specific goal (e.g., GPS navigation)
- **Learning agents**: improve behavior over time (e.g., recommendation systems)

> **Note:** Whenever you see a system that *observes, decides, and acts*, you are looking at an AI agent.

State

A **state** describes the current situation of the agent.

Examples:

- Position in a maze
- Configuration of a puzzle
- Current board in a game

Initial State

The **starting state** of the problem.

Actions

Actions are the **possible moves** an agent can take.

Examples:

- Move up / down / left / right
- Slide a tile
- Place X or O in a square

Transition Model

The **transition model** describes **how the world changes** when an agent takes an action.

In simple words:

> If the agent is in a state and performs an action, the transition model tells us **what the next state will be**.

> **Formal idea:** State + Action → New State

Why is the transition model important?

Without a transition model:

- The AI would not know the consequences of its actions
- The AI could not plan ahead
- Search algorithms would not work

The transition model is what allows AI to **simulate the future** before acting.

Example: Sliding Puzzle Game (8-Puzzle / 15-Puzzle)

Consider a sliding puzzle with numbered tiles and one empty space.

- **State:** the current arrangement of all tiles
- **Action:** slide one tile into the empty space
- **Transition model:** defines how the board changes after the slide

Example scenario:

Current state:

[1 2 3] [4 _ 5] [6 7 8]

Action: **slide tile 5 to the left**

Resulting state (after applying the transition model):

[1 2 3] [4 5 _] [6 7 8]

The transition model answers:

- Which actions are allowed (only tiles next to the empty space)
- What the exact new configuration will be

> **Note:** The transition model is like the *rules of the game* — it tells the AI exactly how the world changes after each move.

Goal Test

The **goal test** is a function that checks **whether the AI has successfully solved the problem**.

In simple words:

> The goal test answers the question: **“Are we done?”**

Why is the goal test important?

Without a goal test:

- The AI would not know when to stop searching
- The AI might continue forever
- The AI could not recognize success

The goal test gives the AI a **clear stopping condition**.

What does a goal test do?

A goal test:

- Takes a **state** as input
- Returns **true** if the state is a goal state
- Returns **false** otherwise

This function is checked **every time a new state is explored**.

Example 1: Maze Solving (Example to Draw on the Board)

This is an **ideal example to draw step by step**.

You can draw a **simple grid maze** like this on the board:

```
S  .  .  #
#  #  .  #
.  .  .  G
```

Legend:

- **S** = Start (initial state)
- **G** = Goal (exit)
- **.** = free cell
- **#** = wall

Steps:

- **State:** the current cell where the agent is located (for example: at **S**)
- **Actions:** move **up, down, left, right** (only if there is no wall)

- **Goal:** reach the cell marked **G**
- **Goal test:**
    - Ask at every step: **"Is my current position G?"**

If YES → solution found (stop searching) If NO → try another move

Note:

- The agent may pass near the goal but **does not stop** unless the goal test is true
- The goal test is checked **after every move**

Example 2: Sliding Puzzle Game

This example is **very effective to draw and explain step by step** because we can easily see the changes.

You can draw a **3×3 sliding puzzle** like this:

```
[ 1  2  3 ]
[ 4  _  5 ]
[ 6  7  8 ]
```

---

steps:

- **State:** the full configuration of the board (the position of every tile, including the empty space _)
- **Actions:** slide a tile **up, down, left, or right** *into the empty space* (only tiles next to _ are allowed to move)
- **Goal:** reach the correct ordered configuration:

```
[ 1  2  3 ]
[ 4  5  6 ]
[ 7  8  _ ]
```

Goal test (what the AI checks):

At every step, the AI asks:

> "Are all tiles in their correct target positions?"

- If **NO** → continue searching and applying actions
- If **YES** → solution found, stop

Note

- The AI may reach a configuration that *looks close* to the goal, but the goal test is still **false**
- The goal test only becomes **true** when the configuration is **exactly correct**
- The AI does not care *how* it arrived there — only *whether* the goal is satisfied

The sliding puzzle goal test is like checking if a lock combination is exactly correct — almost correct is still wrong.

Example 3: Game Playing (Tic-Tac-Toe)

This example is **excellent for explaining goal tests, states, and decision-making** in games.

```
 X | O | X
---+---+---
 O | X |
---+---+---
   | O |
```

** step:**

- **State:** the complete configuration of the board (which cells contain X, which contain O, and which are empty)
- **Players:**
    - Player X (the AI)
    - Player O (the opponent)
- **Actions:** place your symbol (X or O) in any empty cell

Goal in Tic-Tac-Toe:

The goal depends on the player:

- Player X wants to **win**
- Player O wants to **win** (opposite goal)

Goal test (what the AI checks):

After every move, the AI checks:

1. **Win condition**
    - Has X or O formed **three symbols in a row**?
        - Horizontally
        - Vertically
        - Diagonally

2. **Draw condition**
    - Is the board full **with no winner**?

- If a win or draw is detected → **terminal state reached**
- Otherwise → the game continues

Note:

- The goal test does **not** suggest which move to play
- It only tells the AI **whether the game has ended**
- Many states are **non-goal states**, even if the board looks close to a win

---

> In games, the goal test is like a referee — it does not play the game, it only decides when the game is finished and who won.

Path Cost

The **path cost** measures **how expensive a solution is**, from the initial state to the goal state.

In simple words:

> Path cost answers the question: **"How much did it cost me to reach the goal?"**

Why is path cost important?

Without path cost:

- The AI might find *a* solution, but not a **good** one
- The AI would not know which solution is **better** than another

Path cost allows the AI to:

- Compare different solutions
- Prefer **shorter, faster, or cheaper** paths

Example to Draw on the Board: Maze with Costs

Draw the following maze on the board:

```
S  .  .  G
#  #  .  #
.  .  .  .
```

Now explain the **cost of each move**:

- Moving to any neighboring cell costs **1**

Step-by-step explanation

- **Initial state:** S
- **Goal:** G
- **Actions:** move up, down, left, right
- **Path cost:** total number of moves

Now show **two possible paths**:

Path 1 (short path):

- S → right → right → right → G
- Cost = 3

Path 2 (long path):

- S → down → down → right → right → up → up → right → G
- Cost = 7

What does the AI prefer?

- Both paths reach the goal
- But the AI chooses **Path 1**, because:

> **3 < 7**

This is what we mean by the **lowest-cost solution**.

Note

- A solution is not just about reaching the goal
- AI prefers the solution with the **minimum total cost**
- Different problems can define cost differently:
    - Distance
    - Time
    - Money
    - Energy

> Path cost is like choosing the fastest route on Google Maps — not just any route that gets you there.

---

4. State Space and Search Tree

- Each **state** is a node

- Each **action** is an edge
- All possible states form the **state space**

This can be represented as a **graph or tree**.

---

5. Generic Search Algorithm

Basic Idea

1. Start from the initial state
2. Store states to explore in a **frontier**
3. Repeatedly:
    - Remove one state from the frontier
    - Check if it is the goal
    - If not, expand it
4. Stop when the goal is found or no states remain

---

6. Avoiding Infinite Loops

Problem

The agent may revisit the same states repeatedly.

Solution

Maintain an **explored set** of visited states and avoid revisiting them.

---

7. Depth-First Search (DFS)

Depth-First Search (DFS) is a **search strategy** that explores a problem by going **as deep as possible** along one path before trying other alternatives.

In simple words:

> DFS says: **"Let me follow this path until I cannot go further."**

Why do we use DFS?

DFS is used because:

- It is **simple to understand and implement**
- It uses **less memory** than many other search algorithms
- It is useful when:

- The solution is **deep** in the search tree
- Memory is limited

However, DFS is **not guaranteed** to find the best or shortest solution.

Is DFS a one-solution problem?

No — **the problem may have many solutions**, but:

- DFS will usually find **one solution first**
- That solution depends on:
  - The order of actions
  - The structure of the search tree

DFS **does not compare all solutions**. It stops as soon as it finds *a* solution.

How DFS works (conceptually)

DFS uses a **stack (LIFO – Last In, First Out)**:

1. Start from the initial state
2. Go to the first child state
3. Keep going deeper and deeper
4. If no more actions are possible → backtrack
5. Try a different path

Example to Draw on the Board (Maze)

Draw this simple maze:

```
S  A  B  G
|  |  |  |
C  D  E  F
```

Explain it like this:

- **S** = Start
- **G** = Goal
- Each letter is a state

Assume the action order is: **Right first, then Down**.

```
# Depth-First Search (DFS)
# Action order: Right first, then Down
```

```python
# Graph representation of the maze
graph = {
    "S": ["A", "C"],   # Right, then Down
    "A": ["B", "D"],
    "B": ["G", "E"],
    "C": [],
    "D": [],
    "E": [],
    "F": [],
    "G": []
}

start = "S"
goal = "G"

# DFS uses a STACK (LIFO)
stack = [start]
visited = set([start])

print("Initial stack:", stack)

while stack:
    current = stack.pop()
    print(f"\nVisit {current}")

    # Goal test
    if current == goal:
        print("Goal found!")
        break

    # Push neighbors in REVERSE order
    # so that 'Right' is explored before 'Down'
    for neighbor in reversed(graph[current]):
        if neighbor not in visited:
            visited.add(neighbor)
            stack.append(neighbor)

    print("Stack now:", stack)
```

```
Initial stack: ['S']

Visit S
Stack now: ['C', 'A']

Visit A
Stack now: ['C', 'D', 'B']

Visit B
Stack now: ['C', 'D', 'E', 'G']

Visit G
```

```
Goal found!
```

DFS Step-by-Step (what the AI does)

1. Start at **S**
2. Go right to **A**
3. Go right to **B**
4. Go right to **G** → Goal found

DFS **stops immediately** here.

> DFS does not check if there is a shorter or better path.

What if the goal is not on the first path?

Explaination:

- DFS will go as deep as possible
- If it reaches a dead end, it **backtracks**
- Then it tries another branch

This backtracking is a key idea in DFS.

Advantages of DFS

- Uses **less memory**
- Easy to implement
- Can be fast if the goal is deep

Disadvantages of DFS

- May find a **very long solution** even if a short one exists
- Can get stuck in deep or infinite paths (without limits)
- Not optimal (does not guarantee shortest path)

> **Note:** DFS is like exploring a maze by always taking one corridor until you hit a wall, then going back and trying another path.

---

8. Breadth-First Search (BFS)

Breadth-First Search (BFS) is a **search strategy** that explores the state space **level by level**, meaning it first explores all states that are close to the start before going deeper.

In simple words:

> BFS says: **"First explore everything that is 1 step away, then 2 steps away, then 3 steps away."**

Why do we use BFS?

BFS is used because:

- It is **guaranteed to find a solution** if one exists
- It is **guaranteed to find the shortest path** (minimum number of steps)
- It is very intuitive and systematic

This makes BFS ideal for problems where **path cost = number of steps**.

Is BFS a one-solution problem?

No. Like DFS:

- The problem may have **many possible solutions**
- BFS will explore **all shallow solutions first**
- The **first solution found is guaranteed to be the shortest**

So BFS does not just find *a* solution — it finds the **best (shortest) one**.

How BFS works (conceptually)

BFS uses a **queue (FIFO – First In, First Out)**:

1. Start from the initial state
2. Put it in the queue
3. Remove the first state from the queue
4. Expand it (generate all its neighbors)
5. Add new states to the **end of the queue**
6. Repeat until the goal is found

Very Important: Explored Set in BFS

In BFS, maintaining an **explored set (visited states)** is **mandatory**.

Without an explored set:

- The same states would be added to the queue again and again
- The queue would grow very large
- The algorithm could loop forever

So BFS always:

- Marks a state as **visited** when it is first seen
- Never adds the same state twice

Example to Draw on the Board (BFS)

Draw this simple graph:

```
      S
     / \
    A   B
   / \   \
  C   D   G
```

Explain it step by step:

- **S** = Start
- **G** = Goal

BFS Step-by-Step (with Queue)

Initial queue:

```
[S]
```

1. Remove **S**, add its children **A, B**

```
[A, B]
```

2. Remove **A**, add **C, D**

```
[B, C, D]
```

3. Remove **B**, add **G**

```
[C, D, G]
```

4. Remove **C** → not goal
5. Remove **D** → not goal
6. Remove **G** → goal found

BFS stops here.

Why is this solution optimal?

- BFS explored all states at depth 1 and 2 **before** depth 3
- When **G** is found, BFS knows:

> "There is no solution with fewer steps."

Python Implementation: BFS on a Simple Graph (Step-by-Step Queue)

```
from collections import deque

# Define the graph (adjacency list)
# IMPORTANT: The order of neighbors matters for BFS
#
#       S
#      / \
#     A   B
#    / \   \
#   C   D   G

graph = {
    "S": ["A", "B"],
    "A": ["C", "D"],
    "B": ["G"],
    "C": [],
    "D": [],
    "G": []
}

start = "S"
goal = "G"

# BFS uses a QUEUE (FIFO)
queue = deque([start])
visited = set([start])

print("Initial queue:", list(queue))

while queue:
    current = queue.popleft()
    print(f"\nRemove {current}")

    # Goal test
    if current == goal:
        print("Goal found!")
        break

    # Add neighbors in the defined order
    for neighbor in graph[current]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)
```

```
    print("Queue now:", list(queue))
```

```
Initial queue: ['S']

Remove S
Queue now: ['A', 'B']

Remove A
Queue now: ['B', 'C', 'D']

Remove B
Queue now: ['C', 'D', 'G']

Remove C
Queue now: ['D', 'G']

Remove D
Queue now: ['G']

Remove G
Goal found!
```

Advantages of BFS

- Always finds the shortest path
- Complete (will find a solution if one exists)
- Very clear and systematic

Disadvantages of BFS

- Uses **a lot of memory**
- Not suitable for very large or infinite state spaces

> **Note:** BFS is like a wave spreading from the start — it reaches the nearest goal first.

---

9. Informed vs Uninformed Search

Search strategies in AI can be divided into **two major categories** based on whether they use extra knowledge about the goal.

This distinction is very important for understanding **why some algorithms are faster and smarter than others**.

Uninformed Search (Blind Search)

Main idea: The AI has **no additional information** about where the goal is.

In simple words:

> The AI explores the state space **without guidance**, only following the problem rules.

Characteristics of Uninformed Search

- The AI only knows:
  - The initial state
  - The possible actions
  - The goal test
- It does **not know**:
  - How close it is to the goal
  - Which direction is better

The search is systematic, but sometimes inefficient.

Examples of Uninformed Search Algorithms

- **Depth-First Search (DFS)**
- **Breadth-First Search (BFS)**

Example to Draw on the Board (Uninformed Search)

Draw this maze:

```
S  .  .  .
#  #  #  .
.  .  .  G
```

Explain:

- The AI starts at **S**
- It does not know where **G** is
- It explores according to the algorithm rules (DFS or BFS)
- It may explore many useless paths before finding the goal

> **Note:** Uninformed search is like exploring a dark room without a map.

Informed Search (Heuristic Search)

Main idea: The AI uses **extra information (heuristics)** to guide the search toward the goal.

In simple words:

> The AI has a **hint** about where the goal might be.

What is a heuristic?

A **heuristic** is a function that:

- Estimates how close a state is to the goal
- Is fast to compute
- Is not always exact, but useful

Characteristics of Informed Search

- The AI prefers states that **look closer to the goal**
- The search is usually **much faster**
- Fewer states are explored

However:

- A bad heuristic can mislead the AI

Examples of Informed Search Algorithms

- **Greedy Best-First Search**
- *A Search**

Example to Draw on the Board (Informed Search)

Use the same maze:

```
S  .  .  .
#  #  #  .
.  .  .  G
```

Now explain:

- The AI estimates the distance from each cell to **G**
- It always chooses the cell that looks closest to **G**
- Fewer unnecessary paths are explored

> **Note:** Informed search is like having a map and a compass.

Key Comparison

Aspect	Uninformed Search	Informed Search
Uses heuristic?	No	Yes
Knows goal direction?	No	Approximately
Speed	Slower	Faster
Explores many states?	Yes	Fewer

> **Final takeaway:** Informed search uses knowledge to search **smarter**, not harder.

---

10. Heuristics

A **heuristic** is a function that estimates **how close a state is to the goal**.

In simple words:

> A heuristic is an **educated guess** that helps the AI decide *which direction looks better*.

Why do we need heuristics?

Without heuristics:

- The AI explores many unnecessary states
- The search can be very slow
- The AI behaves blindly (like DFS or BFS)

Heuristics help the AI:

- Focus on promising paths
- Reach the goal faster
- Explore fewer states

> **Key idea:** Heuristics make search **smarter**, not random.

Important properties of heuristics

A good heuristic is:

- **Fast** to compute
- **Easy** to understand
- **Reasonably accurate** (but not perfect)

A heuristic does **not** have to be exact. It only needs to be **useful**.

Example to Draw on the Board: Manhattan Distance

This is the most common heuristic for grid-based problems (mazes, puzzles).

Manhattan Distance = number of horizontal steps + number of vertical steps

---

Board Example (Step by Step)

Draw this grid:

```
S  .  .  .
.  .  .  .
.  .  .  G
```

Assume:

- **S** = current state
- **G** = goal

Now count:

- Horizontal distance = 3
- Vertical distance = 2

So:

```
h(S) = 3 + 2 = 5
```

This means:

> "From here, it looks like we are about 5 steps away from the goal."

How the heuristic is used

At every step, the AI:

1. Computes the heuristic value for each possible next state
2. Compares these values
3. Prefers the state with the **smallest heuristic value**

Important note

- A heuristic **does not guarantee** correctness by itself
- It only provides **guidance**
- The final correctness comes from the search algorithm

Good vs Bad Heuristic (Intuition)

- **Good heuristic:** roughly points toward the goal
- **Bad heuristic:** misleading or random

> **Note:** A heuristic is like asking someone for directions — they may not be perfect, but they usually help you get closer.

---

11. Greedy Best-First Search

Greedy Best-First Search is an **informed search algorithm** that always expands the node that **looks closest to the goal** according to a heuristic.

In simple words:

> Greedy search says: **"Go to the place that seems closest to the goal right now."**

Why is it called *Greedy*?

It is called *greedy* because:

- It makes a **local decision** at each step
- It chooses what looks best **right now**
- It does **not think about the total path cost**

The algorithm is focused only on the heuristic value **h(n)**.

How Greedy Best-First Search works

At every step:

1. Look at all possible next states
2. Compute the heuristic value **h(n)** for each state
3. Choose the state with the **smallest h(n)**
4. Expand that state first

It **ignores**:

- How long the path already is
- Whether another path might be cheaper overall

Important Formula (Conceptual)

Greedy Best-First Search uses:

```
f(n) = h(n)
```

Only the heuristic matters.

Example to Draw on the Board (Maze)

Draw this grid:

```
S  .  .  .
#  #  #  .
.  .  .  G
```

Now explain step by step:

- **S** = Start
- **G** = Goal
- The heuristic is **Manhattan distance** to G

Step-by-Step Explanation

1. From **S**, list all possible next states (legal moves).
2. For each next state, compute the heuristic value **h(n)** (Manhattan distance to **G**).
3. Choose the state with the **smallest h(n)**.
4. Move to that state and mark it as visited.
5. Repeat the same process until **G** is reached.

Final Solution Found by Greedy Search (Example Outcome)

For the maze shown above, the open cells force the search to go along the **top row** and then **down the last column**. A correct example path is:

```
S → right → right → right → down → down → G
```

This is a **valid solution**, because the goal is reached.

```
import heapq

# Maze definition
maze = [
    ['S', '.', '.', '.'],
    ['#', '#', '#', '.'],
    ['.', '.', '.', 'G']
]

rows = len(maze)
cols = len(maze[0])

# Find start and goal
```

```
for i in range(rows):
    for j in range(cols):
        if maze[i][j] == 'S':
            start = (i, j)
        if maze[i][j] == 'G':
            goal = (i, j)

# Possible moves: Right, Down, Left, Up
moves = [(0,1), (1,0), (0,-1), (-1,0)]

# Priority queue (min-heap): (h(n), state)
frontier = []
heapq.heappush(frontier, (manhattan_distance(start, goal), start))

visited = set()
visited.add(start)

print("Start:", start)
print("Goal:", goal)

step = 0

while frontier:
    step += 1
    print(f"\n--- Step {step} ---")
    print("Frontier (priority queue):", frontier)

    h, current = heapq.heappop(frontier)
    print(f"Expanding node {current} with h(n) = {h}")

    # Goal test
    if current == goal:
        print("\nGoal reached!")
        break

    x, y = current

    for dx, dy in moves:
        nx, ny = x + dx, y + dy

        if 0 <= nx < rows and 0 <= ny < cols:
            if maze[nx][ny] != '#' and (nx, ny) not in visited:
                visited.add((nx, ny))
                h_value = manhattan_distance((nx, ny), goal)
                print(f"  Adding {(nx, ny)} with h(n) = {h_value}")
                heapq.heappush(frontier, (h_value, (nx, ny)))
```

```
Start: (0, 0)
Goal: (2, 3)

--- Step 1 ---
```

```
Frontier: [(5, (0, 0))]
Expanding node (0, 0) with h(n) = 5
  Adding (0, 1) with h(n) = 4

--- Step 2 ---
Frontier: [(4, (0, 1))]
Expanding node (0, 1) with h(n) = 4
  Adding (0, 2) with h(n) = 3

--- Step 3 ---
Frontier: [(3, (0, 2))]
Expanding node (0, 2) with h(n) = 3
  Adding (0, 3) with h(n) = 2

--- Step 4 ---
Frontier: [(2, (0, 3))]
Expanding node (0, 3) with h(n) = 2
  Adding (1, 3) with h(n) = 1

--- Step 5 ---
Frontier: [(1, (1, 3))]
Expanding node (1, 3) with h(n) = 1
  Adding (2, 3) with h(n) = 0

--- Step 6 ---
Expanding node (2, 3) with h(n) = 0

Goal reached!
```

Important Observation

- Greedy search chooses the next state using only **h(n)** (estimated closeness to the goal).
- In **this specific maze**, there is essentially **one main valid route**, so Greedy reaches the goal quickly.
- In other mazes (with multiple routes), Greedy can still reach the goal but may choose a route that is **not the shortest**, because it ignores the path cost already paid.

> **takeaway:** Greedy Best-First Search can be very fast, but it is not guaranteed to find the **shortest** solution.

Why can Greedy Search fail?

Greedy search can make **bad decisions** because:

- It does not consider obstacles well
- It ignores the cost already paid
- It can get stuck going around walls

Example of a Wrong Choice

This is the **most important example to show why Greedy search can fail**.

Example to Draw on the Board

Draw the following maze:

```
S  .  .  .  G
#  #  #  .  #
.  .  .  .  .
```

Legend:

- **S** = Start
- **G** = Goal
- **.** = free cell
- **#** = wall

What the heuristic says

Assume the heuristic used is **Manhattan distance to the goal (G)**.

At the start state **S**:

- The agent evaluates **only legal moves**.
- Moving **down** is **not allowed** because there is a wall (#) directly below **S**.
- Moving **right** is the **only legal move**, and it also reduces the Manhattan distance to **G**.

Therefore, Greedy Best-First Search has **no alternative choice** at the beginning and must move:

```
S → right
```

After this first move, Greedy continues to prefer moves along the **top row**, because each rightward step:

- Is legal
- Further reduces the heuristic value **h(n)**

This behavior highlights an important property of Greedy search: it **follows the heuristic whenever possible**, without considering future obstacles or the total cost already accumulated.

What actually happens

As the search continues:

- The path along the top row eventually **hits a wall and a dead-end** near the goal.
- Because Greedy has already committed to this direction, it must take a **long detour** to bypass the obstacle.

A possible greedy path is:

```
S → right → right → right → down → down → left → left → left → right → right → G
```

This path **reaches the goal**, but with a **high path cost**.

A better (shorter) solution

A shorter solution exists, but it initially looks worse according to the heuristic:

```
S → down → down → right → right → right → right → G
```

This path:

- Does not reduce the heuristic as quickly at the beginning
- Results in a **lower total path cost**

Greedy search fails to choose this path because it ignores the cost already paid.

Key lesson

- Greedy Best-First Search considers **only h(n)** (estimated closeness to the goal)
- It ignores **g(n)** (the cost accumulated so far)
- As a result, it can make **locally good but globally poor decisions**

> **Note:** Greedy search is like walking straight toward a destination because it looks close, only to discover a fence that forces a long detour.

Advantages of Greedy Best-First Search

- Very fast in practice
- Explores relatively few states
- Simple to understand and implement

Disadvantages of Greedy Best-First Search

- Not optimal (does not guarantee the shortest path)
- Not complete in all state spaces
- Highly dependent on the quality of the heuristic

> **Final takeaway:** Greedy Best-First Search is efficient and intuitive, but its lack of long-term planning motivates the need for more robust algorithms such as **A***.

---

12. A* Search Algorithm

Core Idea

A* Search improves upon Greedy search by considering **both**:

- The cost already paid (**g(n)**)
- The estimated cost to the goal (**h(n)**)

In simple words:

> **A*** says: *"Choose the path that is cheap so far AND still looks promising."*

Evaluation Function

```
f(n) = g(n) + h(n)
```

Where:

- **g(n)** = cost from the start to the current state
- **h(n)** = heuristic estimate from the current state to the goal
- **f(n)** = total estimated cost of the solution through this state

A* always expands the state with the **smallest f(n)** value.

Example to Draw on the Board (Very Simple)

Draw this grid:

```
S  .  .
.  .  .
.  .  G
```

Assume:

- Each move costs **1**
- Heuristic **h(n)** = Manhattan distance to **G**

Step-by-Step A* Explanation

Step 1: Start at S

- g(S) = 0
- h(S) = 4
- f(S) = 0 + 4 = 4

Step 2: Expand neighbors of S

For each neighbor, compute g, h, f:

- Right:
  - g = 1
  - h = 3
  - f = 4
- Down:
  - g = 1
  - h = 3
  - f = 4

A* can choose either (same f value).

Step 3: Continue expanding lowest f(n)

At each step:

- g(n) increases by 1
- h(n) decreases as we get closer to G
- f(n) stays minimal for good paths

A* avoids paths that:

- Are already expensive (large g)
- Or look far from the goal (large h)

Final Solution Found by A*

One optimal solution path is:

```
S → right → right → down → down → G
```

- Total cost = 4
- This is the **shortest possible path**

Why this example is important

This example shows clearly that:

- Greedy uses **only h(n)**
- A* uses **g(n) + h(n)**
- A* avoids long detours
- A* guarantees the shortest path (with a good heuristic)

> **Note:** A* is like Google Maps — it considers how far you have already driven *and* how far you still need to go.

Key Properties of A*

- Complete (will find a solution if one exists)
- Optimal (with an admissible heuristic)
- Efficient in practice

```
import heapq

# Grid definition
grid = [
    ['S', '.', '.'],
    ['.', '.', '.'],
    ['.', '.', 'G']
]

rows = len(grid)
cols = len(grid[0])

# Locate start and goal
for i in range(rows):
    for j in range(cols):
        if grid[i][j] == 'S':
            start = (i, j)
        if grid[i][j] == 'G':
            goal = (i, j)

# Moves: Right, Down, Left, Up
moves = [(0,1), (1,0), (0,-1), (-1,0)]

# Priority queue elements: (f(n), g(n), state)
frontier = []
heapq.heappush(frontier, (manhattan_distance(start, goal), 0, start))

visited = set()

step = 0

print("Start:", start)
print("Goal:", goal)

while frontier:
    step += 1
    print(f"\n--- Step {step} ---")
    print("Frontier:")
    for item in frontier:
        print(item)
```

```
    f, g, current = heapq.heappop(frontier)
    print(f"\nExpanding {current}")
    print(f"g(n) = {g}, h(n) = {manhattan_distance(current, goal)}, f(n) = {f}")

    # Goal test
    if current == goal:
        print("\nGoal reached!")
        print("Total cost:", g)
        break

    if current in visited:
        continue

    visited.add(current)

    x, y = current

    for dx, dy in moves:
        nx, ny = x + dx, y + dy

        if 0 <= nx < rows and 0 <= ny < cols:
            if (nx, ny) not in visited:
                new_g = g + 1
                new_h = manhattan_distance((nx, ny), goal)
                new_f = new_g + new_h

                print(f"  Adding {(nx, ny)}:")
                print(f"    g = {new_g}, h = {new_h}, f = {new_f}")

                heapq.heappush(frontier, (new_f, new_g, (nx, ny)))
```

```
Start: (0, 0)
Goal: (2, 2)

--- Step 1 ---
Expanding (0, 0)
g = 0, h = 4, f = 4
  Adding (0, 1): g=1, h=3, f=4
  Adding (1, 0): g=1, h=3, f=4

--- Step 2 ---
Expanding (0, 1)
g = 1, h = 3, f = 4
  Adding (0, 2): g=2, h=2, f=4
  Adding (1, 1): g=2, h=2, f=4

--- Step 3 ---
Expanding (0, 2)
g = 2, h = 2, f = 4
  Adding (1, 2): g=3, h=1, f=4
```

```
--- Step 4 ---
Expanding (1, 2)
g = 3, h = 1, f = 4
  Adding (2, 2): g=4, h=0, f=4

--- Step 5 ---
Expanding (2, 2)
Goal reached!
Total cost: 4
```

Assignment – Solving a Search Problem from Scratch (Up to A*)

Assignment Title

*From Problem Definition to A Search: Building an Intelligent Path-Finding Agent**

Problem Description

You are given a **grid-based map** representing a small city. An AI agent must move from a **start position (S)** to a **goal position (G)** while avoiding obstacles.

```
S  .  .  .
#  #  .  #
.  .  .  G
```

Legend:

- **S** = Start
- **G** = Goal
- **.** = Free cell
- **#** = Obstacle (cannot be crossed)

The agent can move **up, down, left, right**. Each move has a **cost of 1**.

Apply A* Step by Step

1. For each expanded state, compute:
    - g(n)
    - h(n)
    - f(n) = g(n) + h(n)
2. Show which node A* selects at each step
3. Stop when the goal is reached

Expected Outcome

- A*: finds the **shortest path efficiently**

Submission Requirements

- Python implementation of A*